

A02:2025 Security Misconfiguration



Background.

Moving up from #5 in the previous edition, 100% of the applications tested were found to have some form of misconfiguration, with an average incidence rate of 3.00%, and over 719k occurrences of a Common Weakness Enumeration (CWE) in this risk category. With more shifts into highly configurable software, it's not surprising to see this category moving up. Notable CWEs included are *CWE-16 Configuration* and *CWE-611 Improper Restriction of XML External Entity Reference (XXE)*.

Score table.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Max Coverage	Avg Coverage	Avg Weighted Exploit	Avg Weighted Impact
16	27.70%	3.00%	100.00%	52.35%	7.96	3.97

Description.

Security misconfiguration is when a system, application, or cloud service is set up incorrectly from a security perspective, creating vulnerabilities.

The application might be vulnerable if:

- It is missing appropriate security hardening across any part of the application stack or improperly configured permissions on cloud services.
- Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges).
- Default accounts and their passwords are still enabled and unchanged.
- A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.
- For upgraded systems, the latest security features are disabled or not configured securely.
- Excessive prioritization of backward compatibility leading to insecure configuration.

- The security settings in the application servers, application frameworks (e.g., Struts, Spring, ASP.NET), libraries, databases, etc., are not set to secure values.
- The server does not send security headers or directives, or they are not set to secure values.

Without a concerted, repeatable application security configuration hardening process, systems are at a higher risk.

How to prevent.

Secure installation processes should be implemented, including:

- A repeatable hardening process enabling the fast and easy deployment of another environment that is appropriately locked down. Development, QA, and production environments should all be configured identically, with different credentials used in each environment. This process should be automated to minimize the effort required to set up a new secure environment.
- A minimal platform without any unnecessary features, components, documentation, or samples. Remove or do not install unused features and frameworks.
- A task to review and update the configurations appropriate to all security notes, updates, and patches as part of the patch management process (see [A03 Software Supply Chain Failures](#)). Review cloud storage permissions (e.g., S3 bucket permissions).
- A segmented application architecture provides effective and secure separation between components or tenants, with segmentation, containerization, or cloud security groups (ACLs).
- Sending security directives to clients, e.g., Security Headers.
- An automated process to verify the effectiveness of the configurations and settings in all environments.
- Proactively add a central configuration to intercept excessive error messages as a backup.
- If these verifications are not automated, they should be manually verified annually at a minimum.
- Use identity federation, short-lived credentials, or role-based access mechanisms provided by the underlying platform instead of embedding static keys or secrets in code, configuration files, or pipelines.

Example attack scenarios.

Scenario #1: The application server comes with sample applications not removed from the production server. These sample applications have known security flaws that attackers use to

compromise the server. Suppose one of these applications is the admin console, and default accounts weren't changed. In that case, the attacker logs in with the default password and takes over.

Scenario #2: Directory listing is not disabled on the server. An attacker discovers they can simply list directories. The attacker finds and downloads the compiled Java classes, which they decompile and reverse engineer to view the code. The attacker then finds a severe access control flaw in the application.

Scenario #3: The application server's configuration allows detailed error messages, such as stack traces to be returned to users. This potentially exposes sensitive information or underlying flaws, such as component versions that are known to be vulnerable.

Scenario #4: A cloud service provider (CSP) defaults to having sharing permissions open to the Internet. This allows sensitive data stored within cloud storage to be accessed.

References.

- [OWASP Testing Guide: Configuration Management](#)
- [OWASP Testing Guide: Testing for Error Codes](#)
- [Application Security Verification Standard V13 Configuration](#)
- [NIST Guide to General Server Hardening](#)
- [CIS Security Configuration Guides/Benchmarks](#)
- [Amazon S3 Bucket Discovery and Enumeration](#)
- [ScienceDirect: Security Misconfiguration](#)

List of Mapped CWEs

- [CWE-5 J2EE Misconfiguration: Data Transmission Without Encryption](#)
- [CWE-11 ASP.NET Misconfiguration: Creating Debug Binary](#)
- [CWE-13 ASP.NET Misconfiguration: Password in Configuration File](#)
- [CWE-15 External Control of System or Configuration Setting](#)
- [CWE-16 Configuration](#)
- [CWE-260 Password in Configuration File](#)
- [CWE-315 Cleartext Storage of Sensitive Information in a Cookie](#)
- [CWE-489 Active Debug Code](#)
- [CWE-526 Exposure of Sensitive Information Through Environmental Variables](#)

- [CWE-547 Use of Hard-coded, Security-relevant Constants](#)
- [CWE-611 Improper Restriction of XML External Entity Reference](#)
- [CWE-614 Sensitive Cookie in HTTPS Session Without 'Secure' Attribute](#)
- [CWE-776 Improper Restriction of Recursive Entity References in DTDs \('XML Entity Expansion'\)](#)
- [CWE-942 Permissive Cross-domain Policy with Untrusted Domains](#)
- [CWE-1004 Sensitive Cookie Without 'HttpOnly' Flag](#)
- [CWE-1174 ASP.NET Misconfiguration: Improper Model Validation](#)